

```
#=====
#Random Forest for Fault detection in Wireless Communication
#=====

# Import Libraries
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

import matplotlib.pyplot as plt
```

Step1: Load Dataset

```
import pandas as pd

data = pd.read_csv("/content/wireless_signal_quality_dataset.xlsx - Sheet1.csv")

print("First Five Records")
data.head()

print("\nDataset Shape:")
print(data.shape)
```

First Five Records

Dataset Shape:
(100, 10)

Step2: Separate Features and Target

```
X = data.drop('Quality', axis=1)
Y = data['Quality']
```

Step3: Encode Target Labels

```
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()
y = encoder.fit_transform(Y) # Use Y instead of y as y was not defined

print("\nClasses:")
print(encoder.classes_)
```

```
Classes:
['Excellent' 'Fair' 'Good' 'Poor']
```

Step4: Split Dataset

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42)
```

Step5: Build Random Forest Mode

```
from sklearn.ensemble import RandomForestClassifier

from sklearn.tree import DecisionTreeClassifier

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
```

▼ RandomForestClassifier ⓘ ?

```
RandomForestClassifier(random_state=42)
```

Step6 Prediction

```
y_pred = rf.predict(X_test)
```

Step7: Accuracy

```
from sklearn.metrics import accuracy_score
from sklearn.tree import DecisionTreeClassifier
```

```
accuracy = accuracy_score(y_test, y_pred)

print("\nAccuracy = {:.2f}%".format(accuracy*100))
```

Accuracy = 75.00%

Step8: Classification Report

```
from sklearn.metrics import classification_report

print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    target_names=[str(c) for c in encoder.classes_]
))
```

```
Classification Report:
              precision    recall  f1-score   support

 Excellent      0.00      0.00      0.00         1
      Fair      0.57      0.67      0.62         6
      Good      1.00      0.33      0.50         3
      Poor      0.83      1.00      0.91        10

 accuracy              0.75         20
 macro avg      0.60      0.50      0.51         20
 weighted avg   0.74      0.75      0.71         20
```

```
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

Step9: Confusion Matrix

```
from sklearn.metrics import confusion_matrix

cm=confusion_matrix(y_test,y_pred)
print(cm)
```

```
[[ 0  1  0  0]
 [ 0  4  0  2]
 [ 0  2  1  0]
 [ 0  0  0 10]]
```

Step:10 Feature Importance

```
importance = pd.Series(  
    rf.feature_importances_,  
    index=X.columns  
)  
  
importance = importance.sort_values(ascending=False)  
print("\nFeature Importance")  
  
print(importance)
```

Feature Importance	
Signal_Strength_dBm	0.395390
SNR_dB	0.178274
Estimated_Throughput_Mbps	0.164952
Interference_dB	0.073897
Noise_Floor_dBm	0.054547
Distance_m	0.052557
Connected_Devices	0.047600
Channel_Bandwidth_MHz	0.019887
Frequency_GHz	0.012896
dtype: float64	